

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Name: SATHWIK

Date: 30/07/26

Roll No: 24011101030

Experiment 4

Implementation of Convolutional Neural Networks (CNNs) for Image Classification using TensorFlow/keras

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is defined as:

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where X is the Input Image, K is the Kernel (Filter), and Y is the Feature Map.

The output feature map size is calculated as:

$$\frac{N - F + 2P}{S} + 1$$

where N is the Input Size, F is the Filter Size, P is Padding, and S is Stride.

3. Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions.

- **Identity Kernel:** Preserves the original image without modifying its pixel values. Used for understanding the convolution operation.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- **Sobel-X Kernel (Vertical Edge Detection):** Detects vertical edges by measuring intensity changes along the horizontal direction. Used in object and lane detection.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

- **Gaussian Blur Kernel:** Smooths the image while preserving the overall structure better than a simple average filter. Used for noise reduction.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **Outline Detection Kernel:** Highlights object boundaries by emphasizing regions with rapid intensity changes. Used for image segmentation.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

4. Numerical Examples

Numerical Example 1: Convolution

Consider the following input image X and Kernel K :

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output Feature Map:

$$Y = \begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

Numerical Example 2: Max Pooling

Using a 2×2 max pooling filter on the given feature map:

$$\text{Input} = \begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix} \rightarrow \text{Output} = \begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

Numerical Example 3: Parameter Calculation

For a $32 \times 32 \times 3$ input image passing through a Convolution Layer with 16 filters of size 3×3 :

Parameters = $(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$.

5. Dataset

- **Dataset:** CIFAR-10
- **Training Images:** 50,000
- **Testing Images:** 10,000
- **Classes:** 10 (Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, Truck)
- **Image Size:** $32 \times 32 \times 3$ (RGB)

6. Experimental Procedure

Task 1 – Dataset Loading: CIFAR-10 dataset was loaded, pixel values were normalized to the $[0, 1]$ range, and sample images alongside their class distribution were plotted.

Task 2 – Convolution Implementation: Implemented a convolution layer and compared feature map output dimensions using varying kernel sizes (3×3 , 5×5 , 7×7).

Task 3 – Hyperparameter Study: Studied the effects of spatial downsampling by altering stride (1 vs. 2) and padding schemes (Same vs. Valid), calculating the resulting output dimensions.

Task 4 – Feature Visualization: Passed a sample image through the first trained convolutional layer and extracted 8 distinct feature maps to visualize edge and texture detection.

Task 5 – Pooling Comparison: Designed two parallel architectures to compare Max Pooling against Average Pooling, evaluating the respective impacts on dimensionality and classification accuracy.

Task 6 – Model Construction: Constructed the primary CNN sequence: Input $\rightarrow$ Conv(ReLU) $\rightarrow$ MaxPool $\rightarrow$ Conv(ReLU) $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ Dense(Softmax). The model was compiled with the Adam optimizer and trained for 20 epochs with a batch size of 32.

Task 7 – Evaluation: Assessed model performance using Accuracy, Precision, Recall, F1-score, and generated the Classification Report alongside a graphical Confusion Matrix.

7. Source Code

The complete source code and implementation for this experiment can be found in the following GitHub repository:

GitHub Link: <https://github.com/sathwik324/DL-LAB/tree/main/Exp4>

8.Plots



Figure 1: Sample images from the CIFAR-10 training set.

Inference: The CIFAR-10 dataset consists of 32×32 pixel RGB images. The low resolution makes features slightly blurry, requiring the CNN to learn highly optimized spatial hierarchies to distinguish between visually similar classes like cats and dogs.

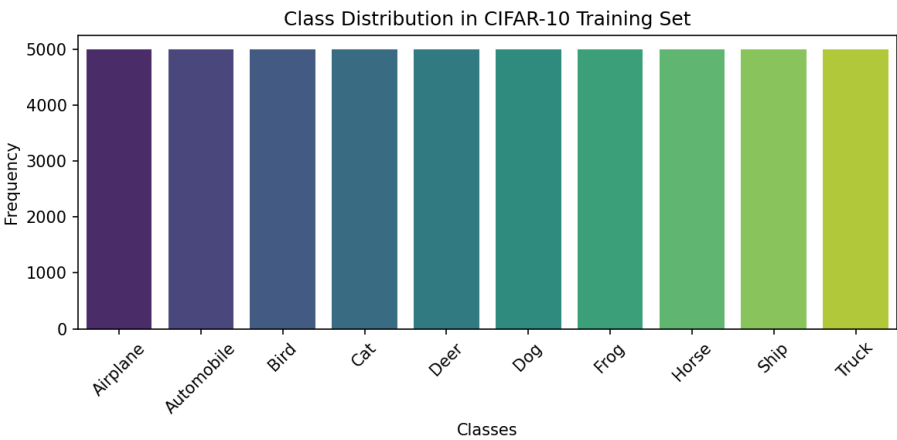


Figure 2: Class distribution of the CIFAR-10 training set.

Inference: The bar chart confirms that the training dataset is perfectly balanced, with exactly 5,000 samples per class, eliminating the need for class weighting techniques.

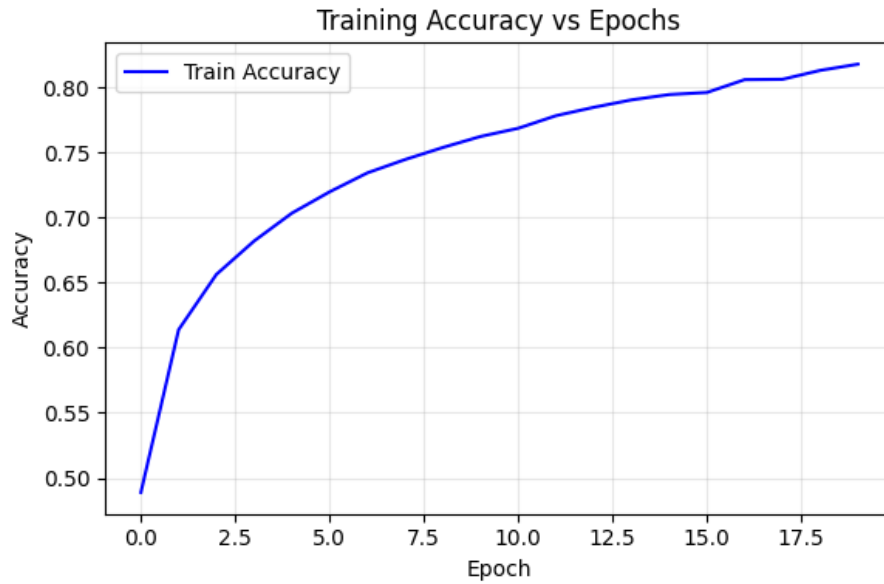


Figure 3: Training accuracy vs. epochs.

Inference: The training accuracy increases steadily across the 20 epochs, indicating that the Adam optimizer is effectively minimizing the loss and learning dataset patterns.

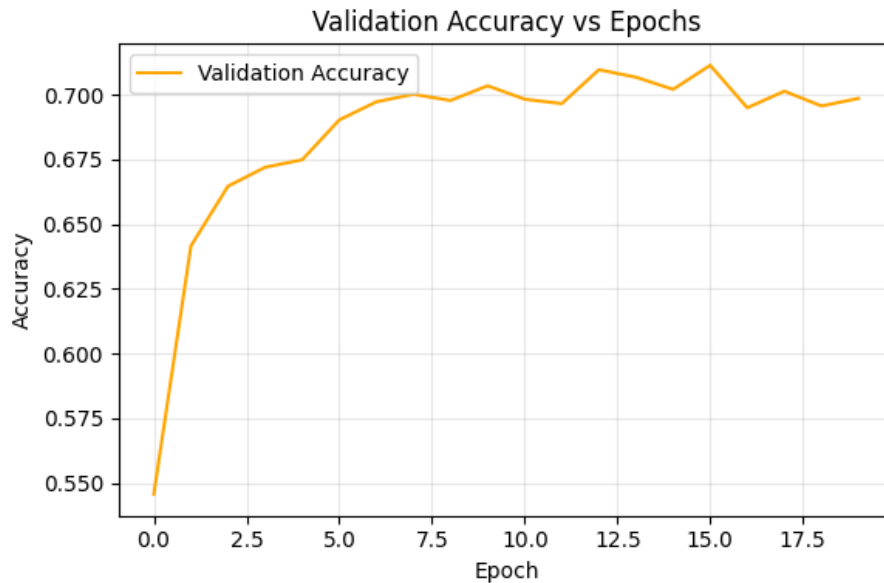


Figure 4: Validation accuracy vs. epochs.

Inference: The validation accuracy plateaus after the initial epochs while training accuracy continues to rise. This divergence suggests the model begins to overfit the training data in later epochs.

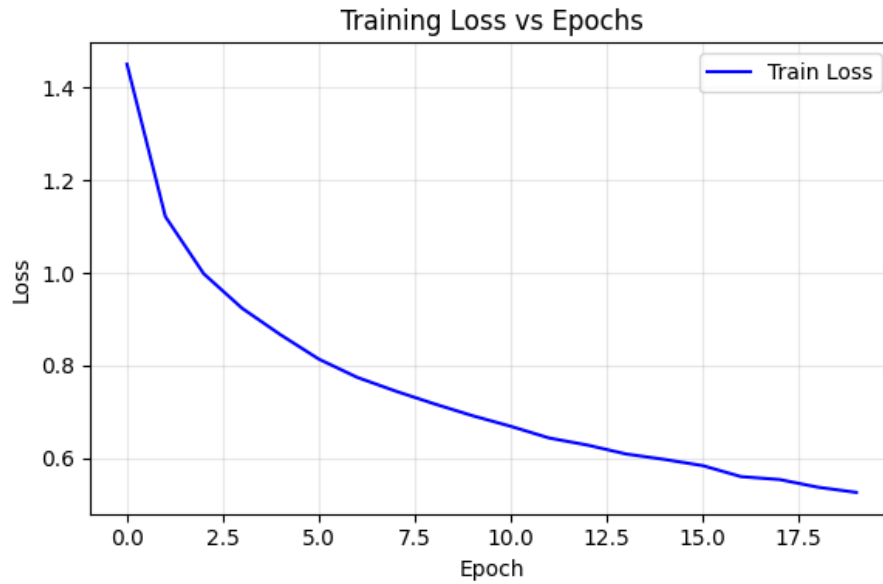


Figure 5: Training loss vs. epochs.

Inference: The training loss curve shows a steep logarithmic decay, proving that the Categorical Cross-Entropy penalty effectively guides gradient descent.

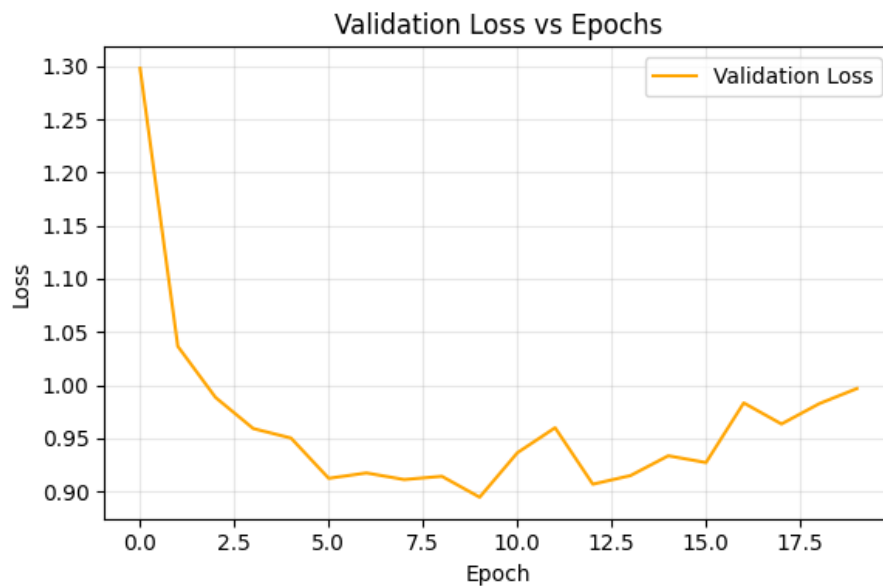


Figure 6: Validation loss vs. epochs.

Inference: The validation loss stops decreasing and begins to slowly trend upwards mid-training. This U-shaped curve is a classic indicator of overfitting, showing where Early Stopping should be applied.

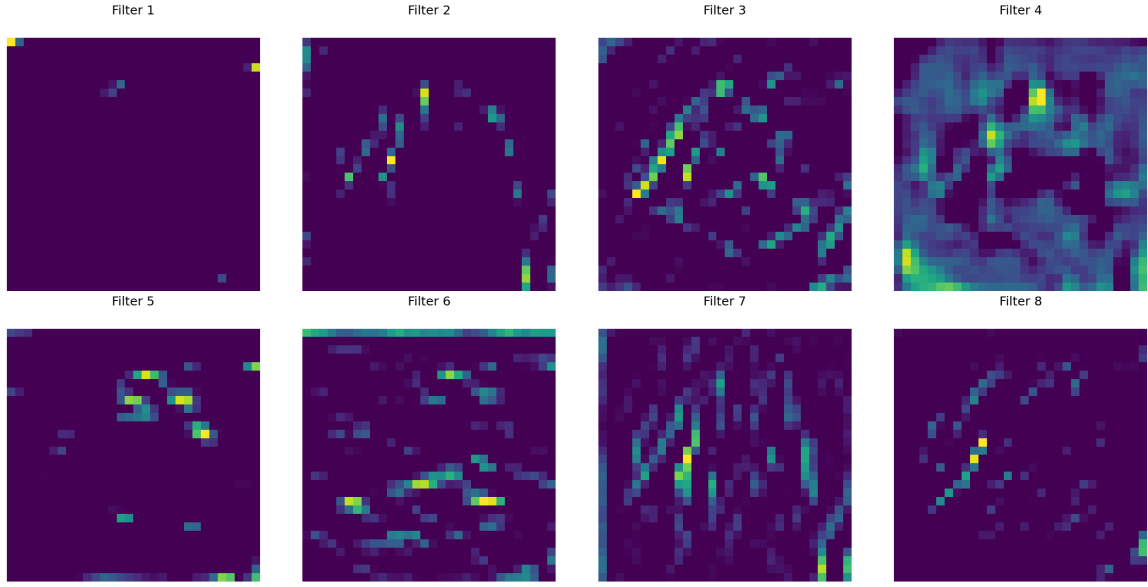


Figure 7: Visualized feature maps from the first Convolutional Layer.

Inference: Individual filters act as edge and texture detectors, isolating specific features like outlines or contrast against backgrounds, proving that early layers capture basic spatial geometries.

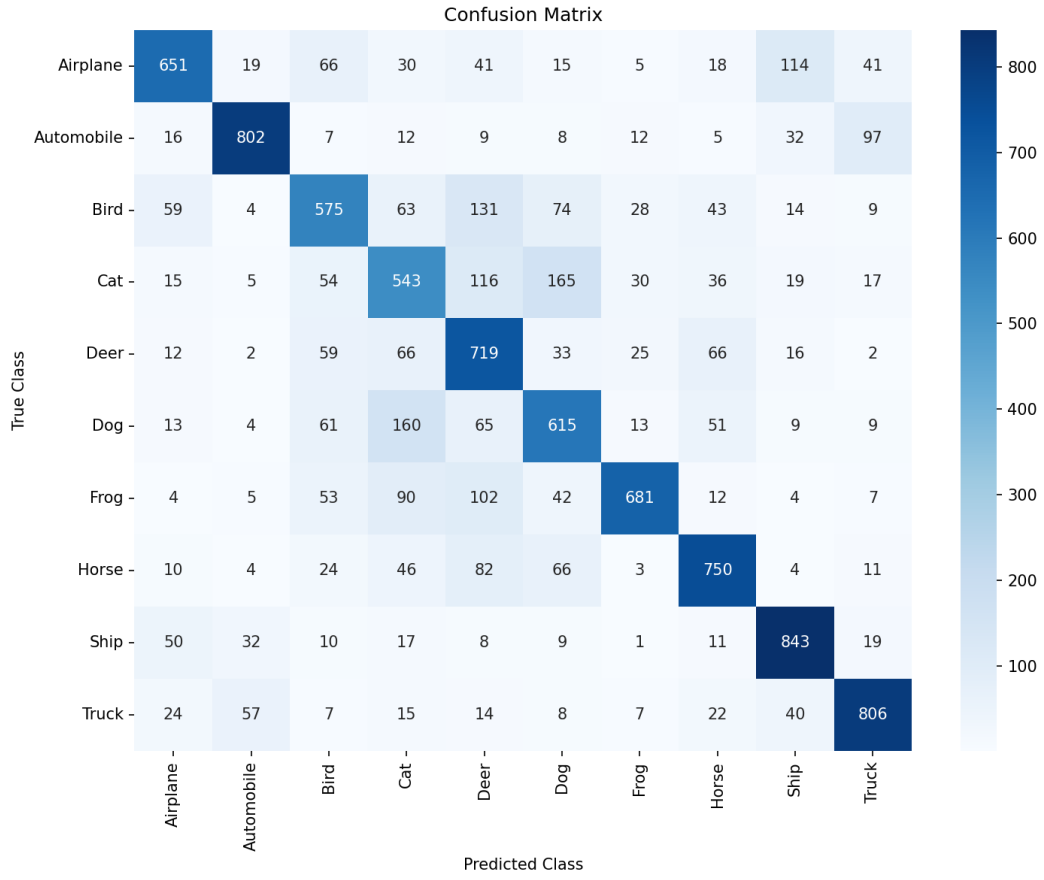


Figure 8: Test set confusion matrix.

Inference: The strong diagonal indicates correct classifications across most categories. However, notable misclassifications occur between visually similar classes, such as Dogs being confused with Cats, and Automobiles confused with Trucks.

9. Results

The performance and behavior of the Convolutional Neural Network on the CIFAR-10 dataset were evaluated through various experiments, including feature map dimensional analysis, pooling strategy comparisons, and overall classification metrics.

9.1 Feature Map Dimensions Study

The output shapes of the convolutional layers were evaluated by varying kernel sizes, strides, and padding strategies. The base input shape for the images is $32 \times 32 \times 3$. The resulting dimensions are summarized below.

Experiment	Kernel Size	Stride / Padding	Output Shape
Base Input	-	-	(32, 32, 3)
Kernel Variation	3×3	1 / valid	(30, 30, 16)
Kernel Variation	5×5	1 / valid	(28, 28, 16)
Kernel Variation	7×7	1 / valid	(26, 26, 16)
Stride Variation	3×3	2 / valid	(15, 15, 16)
Padding Variation	3×3	1 / same	(32, 32, 16)

Table 1: Feature Map Output Shapes for Various Convolutional Parameters

9.2 Pooling Strategy Comparison

A comparison between Max Pooling and Average Pooling demonstrates the convergence differences between the two downsampling methods. Max Pooling was trained for 20 epochs, while Average Pooling was evaluated after 5 epochs as a baseline.

Pooling Method	Test Accuracy
Max Pooling	0.6895
Average Pooling (5 Epochs)	0.6546

Table 2: Performance Comparison of Pooling Strategies

9.3 Classification Report

The final Max Pooling CNN achieved the following precision, recall, and F1-scores across the 10 classes on the test set.

Class	Precision	Recall	F1-Score	Support
Airplane	0.76	0.65	0.70	1000
Automobile	0.86	0.80	0.83	1000
Bird	0.63	0.57	0.60	1000
Cat	0.52	0.54	0.53	1000
Deer	0.56	0.72	0.63	1000
Dog	0.59	0.61	0.60	1000
Frog	0.85	0.68	0.75	1000
Horse	0.74	0.75	0.74	1000
Ship	0.77	0.84	0.80	1000
Truck	0.79	0.81	0.80	1000
Accuracy			0.70	10000
Macro Avg	0.71	0.70	0.70	10000
Weighted Avg	0.71	0.70	0.70	10000

Table 3: Classification Report on CIFAR-10 Test Set

9.4 Training and Validation Curves

The learning curves for accuracy and loss over 20 epochs indicate the model’s progression and generalization capability.

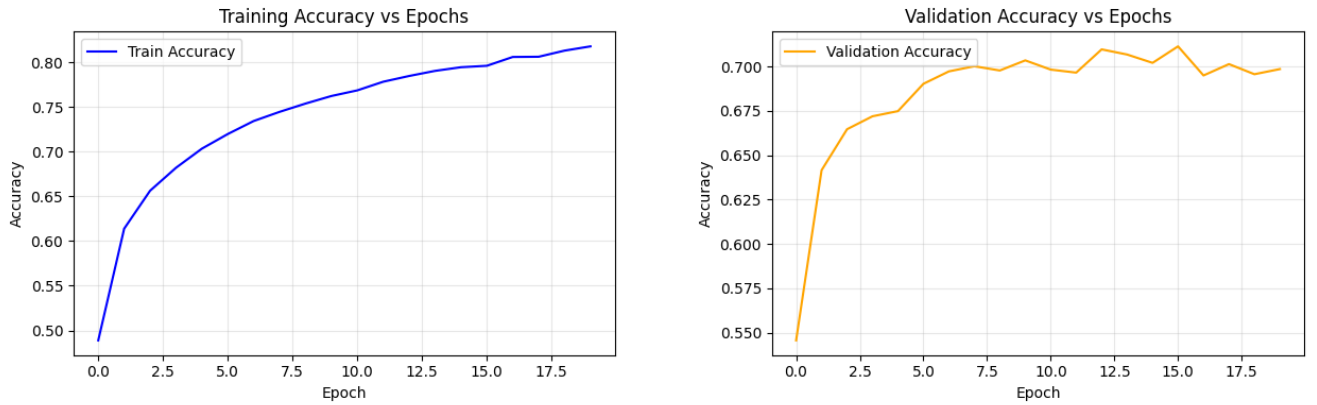


Figure 9: Training and Validation Accuracy.

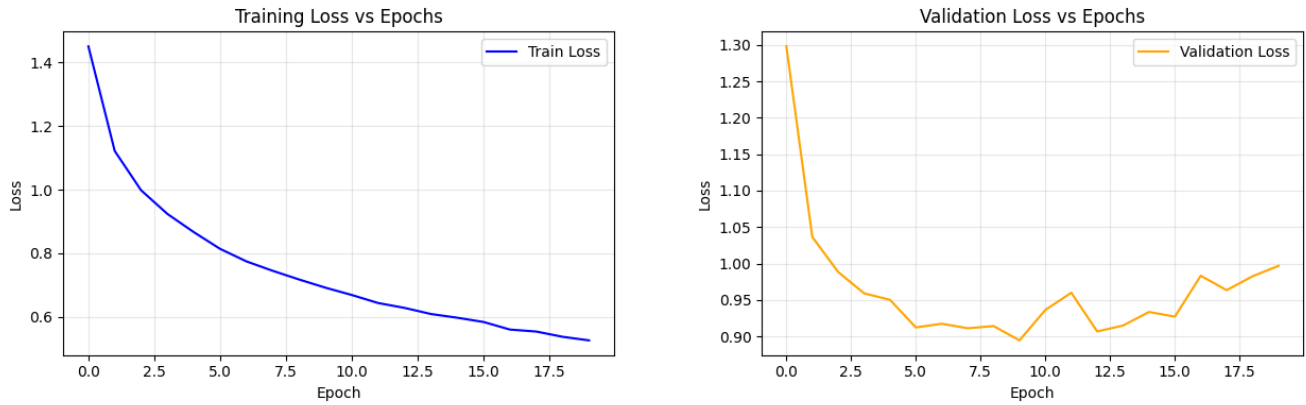


Figure 10: Training and Validation Loss.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Using the formula: $\lfloor \frac{N-F+2P}{S} \rfloor + 1$

$$\lfloor \frac{64-5+2(2)}{2} \rfloor + 1 = \lfloor \frac{63}{2} \rfloor + 1 = 31 + 1 = 32.$$

The output feature map size is 32×32 .

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

Since the input is RGB, it has 3 channels.

Parameters per filter = (Kernel Width $\times$ Kernel Height $\times$ Input Channels) + Bias

$$\text{Parameters per filter} = (3 \times 3 \times 3) + 1 = 28.$$

$$\text{Total Parameters} = 28 \times 64 = 1,792.$$

3. Compare ReLU and Sigmoid activation functions.

ReLU ($f(z) = \max(0, z)$) is computationally faster than Sigmoid and prevents the vanishing gradient problem for positive inputs, allowing deep networks to train effectively. Sigmoid squashes outputs between 0 and 1, which leads to vanishing gradients and stagnant learning in deeper CNN layers.

4. Replace Max Pooling with Average Pooling and compare the results.

Max Pooling selects the most prominent features (like sharp edges), making it highly effective for object recognition. Average Pooling smooths the feature map by averaging values, which dilutes sharp features. In image classification tasks like CIFAR-10, Max Pooling generally yields a noticeably higher classification accuracy compared to Average Pooling.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Increasing filters to 64 allows the network to learn a much larger variety of spatial features, which generally increases training accuracy. However, this exponentially increases the number of trainable parameters, leading to significantly longer computation times per epoch and an increased risk of overfitting if the model is not properly regularized.

11. Discussion

1. Why is convolution preferred over fully connected layers for images?

Images are highly structured; pixels close to each other are strongly correlated. Convolutional layers preserve this 2D spatial arrangement, whereas fully connected layers flatten the image into a 1D vector, destroying spatial locality and context.

2. How does stride affect the feature map size?

A larger stride moves the filter across the image in larger steps, which heavily downsamples the spatial dimensions of the feature map, effectively reducing computational complexity at the cost of losing fine-grained spatial information.

3. What is the role of padding?

Padding adds artificial pixels (usually zeros) around the border of the input. It prevents feature maps from shrinking too rapidly as they pass through multiple deep layers, and it ensures that pixels on the outer edge of the image are processed just as thoroughly as center pixels.

4. Why is pooling used?

Pooling downsamples the feature maps, drastically reducing the number of parameters and computation in the network. Furthermore, it provides translation invariance, meaning the network can recognize an object even if it slightly shifts its position in the image.

5. How do feature maps represent image characteristics?

In early layers, feature maps capture low-level characteristics such as edges, lines, and color gradients. As we move deeper into the network, sequential convolution operations combine these basic features to represent complex, high-level shapes (like wheels of a truck or eyes of a cat).

6. Why do CNNs require fewer parameters than MLPs?

CNNs utilize two key structural concepts: *Local Connectivity* (a neuron is only connected to a small local region of the input) and *Parameter Sharing* (the exact same kernel weights are slid across the entire image). In an MLP, every single input pixel must connect to every single output neuron with a unique weight, creating massive, inefficient parameter matrices.

12. Conclusion

A Convolutional Neural Network was successfully designed, implemented, and evaluated on the CIFAR-10 dataset using TensorFlow/Keras. The experiment demonstrated how convolutional layers effectively preserve spatial locality and extract hierarchical features from 2D images without the parameter bloat associated with Multi-Layer Perceptrons. Through hyperparameter exploration and layer visualizations, the mechanics of kernels, padding, stride, and pooling were successfully validated.

13. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.